WE03

Troubleshooting Apache configurations

The purpose of this presentation is to teach you about some of the useful tools in troubleshooting Apache configurations, and teach some of the techniques for tracking down misconfigurations.

There are but very few personal problems that can't be solved by application of an appropriate amount of dynamite!

Does this apply to Apache?

It might, but it is not among the techniques we'll cover in this presentation.
Much of the inspiration for this talk has been gathered over the last two years in the trenches of #apache. It has been my experience that a vast majority of the problems presented there could have been solved easily by using a few simple tools and techniques.

Doesn't work!

Look, buddy, "doesn't work" is a strong statement.
Does it sit on the couch all day?
Is it making faces at you?
Does it want more money?
Please be specific!

Much can be done in the way of solving a problem by simply asking yourself a question like:

What, exactly, is happening, and how does this differ from what you expect to be happening?

Always remember that the first error message you see in your browser often cannot be trusted. Browsers like Internet Explorer make tracking down problems much harder, by using its own version of error messages. On the Mozillafront theres some agressive caching schemes in play, which will often prevent you from seeing any changes to your Apache configuration without restarting the browser. And to top that off, any sane

server will limit the amount of debugging information it gives away – simply to avoid passing more info to "the enemy" than it really needs to.

Tools

There's a number of tools that are very practical – here is a short presentation of a few of them.

httpd

Not really a tool in itself, but still a good source of information, that can come in handy at a later time.

```
httpd
        -V      : show compile settings
        -t      : run syntax check for config files
        -l      : list compiled in modules
        -L      : list available configuration directives
        -S      : show parsed settings (vhost settings)
```

always test new configurations with *httpd -t*, which will either return "Syntax ok" or report a syntax error and a line number. There's really no need to get caught with a webserver that can't restart, just because you messed up on the syntax.

httpd -l will list compiled in modules and tell you wether there is suexec support (1.3?). This is the place to check wether more modules can be added (mod_so), or Apache has to be recompiled to add more modules. NB! dso modules will not be listed – to find those, check SERVER_ROOT/libexec (1.3) or SERVER_ROOT/modules (2.x)

httpd -L is more of a curiosity, but can be useful in finding configuration directives that you didn't know about before.

httpd -S is useful showing the current virtualHost configuration as Apache parses it.

VirtualHost configuration:

10.0.0.1:80 is a NameVirtualHost

default server example.com (/data/usr/local/apache/conf/httpd.conf:1289)

port 80 namevhost www.example.com (/data/usr/local/apache/conf/httpd.conf:1289)

port 80 namevhost w3.example.com (/data/usr/local/apache/conf/httpd.conf:1299)

As can be seen from the sample output, there's even a line number to refer to the right line of the configuration file.

httpd -V lists some of the compile settings that were used to build:

```
Server version: Apache/2.1.0-dev
```

```
Server built:   Oct  4 2003 13:02:55
Server's Module Magic Number: 20030821:0
Architecture:   32-bit
Server compiled with....
 -D APACHE_MPM_DIR="server/mpm/worker"
 -D APR_HAS_SENDFILE
 -D APR_HAS_MMAP
 -D APR_HAVE_IPV6 (IPv4-mapped addresses enabled)
 -D HTTPD_ROOT="/opt/httpd-2.1"
 -D SUEXEC_BIN="/opt/httpd-2.1/bin/suexec"
 -D DEFAULT_SCOREBOARD="logs/apache_runtime_status"
 -D DEFAULT_ERRORLOG="logs/error_log"
 -D AP_TYPES_CONFIG_FILE="conf/mime.types"
 -D SERVER_CONFIG_FILE="conf/httpd.conf"
```

Most notably, it gives us the version, mpm, HTTPD_ROOT (ServerRoot) and default locations for configuration files and error logs. Additional information could also be extracted from SERVER_ROOT/bin/apxs and SERVER_ROOT/build/config.nice (in 2.1.x – previous versions leaves config.nice in the build tree).

Tools – telnet

As mentioned earlier, browsers lie! Good old telnet can be a way to steer clear of any misleading symptoms from the browser.

Example 1:

```
$ telnet www.apache.org 80
Trying 209.237.227.195...
Connected to www.apache.org.
Escape character is '^]'.
HEAD / HTTP/1.0

HTTP/1.1 200 OK
Date: Thu, 02 Oct 2003 19:21:13 GMT
Server: Apache/2.0.48-dev (Unix)
Accept-Ranges: bytes
Cache-Control: max-age=86400
Expires: Fri, 03 Oct 2003 19:21:13 GMT
Content-Length: 10036
Connection: close
Content-Type: text/html; charset=ISO-8859-1
```

Example 2:

```
$ telnet www.apache.org 80
Trying 209.237.227.195...
Connected to www.apache.org.
Escape character is '^]'.
GET /foundation HTTP/1.0
Host: www.apache.org
```

```
HTTP/1.1 301 Moved Permanently
Date: Fri, 03 Oct 2003 09:49:35 GMT
Server: Apache/2.0.48-dev (Unix)
Location: http://www.apache.org/foundation/
Content-Length: 325
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
```

As can be seen above, emulating a browser with telnet is a simple task, and can easily be expanded to use other headers like User-Agent and Basic auth. There are other tools that can do similar things, but telnet is very practical in being widely available and very often installed by default.

Tools - openssl s_client

openssl s_client can be used very much like telnet, but for testing https connections instead.

```
$ openssl s_client -connect 127.0.0.1:443

-connect host:port
-showcerts
-state
-debug
-cert certname  -key keyfile  -CAfile file

-ssl2, -ssl3, -tls1, -no_ssl2, -no_ssl3, -no_tls1
-cipher cipherlist
-reconnect
```

The basic option is *-connect host:port* which simply tells openssl to connect to a specific ip and port. *-showcerts*will show any certificates that the server sends, allowing you to verify that the correct certificates get sent. The options *-state* and *-debug* add additional information about the transaction. Common errors are errno 26 (connection refused) or errno=29 (connection timed out).

```
$ openssl s_client -connect HOST:PORT -debug

CONNECTED(00000003)
write to 08158160 [081581A8] (130 bytes => 130 (0x82))
0000 - 80 80 01 03 01 00 57 00-00 00 20 00 00 16 00 00   ......W... .....
0010 - 13 00 00 0a 07 00 c0 00-00 66 00 00 07 00 00 05   .........f......
0020 - 00 00 04 05 00 80 03 00-80 01 00 80 08 00 80 00   ................
0030 - 00 65 00 00 64 00 00 63-00 00 62 00 00 61 00 00   .e..d..c..b..a..
0040 - 60 00 00 15 00 00 12 00-00 09 06 00 40 00 00 14   `...........@...
0050 - 00 00 11 00 00 08 00 00-06 00 00 03 04 00 80 02   ................
0060 - 00 80 1c 42 8e db df 85-34 b5 f6 4c 1e e7 cb 57   ...B....4..L...W
0070 - b2 ef 99 21 91 92 1e 08-3f 75 84 16 ce f7 88 63   ...!....?u.....c
0080 - f4 5a                                             .Z
```

```
read from 08158160 [0815D708] (7 bytes => 7 (0x7))
0000 - 3c 21 44 4f 43 54 59                              <!DOCTY
17300:error:140770FC:SSL routines:SSL23_GET_SERVER_HELLO:unknown
protocol:s23_clnt.c:470:
```

The above is an example of running openssl s_client with *-debug*.

Quiz: what is wrong? Answer[1]

Tools - netstat

Netstat is a tool that will print information about network connections.

```
$ netstat -an
Active Internet connections (servers and established)

Proto Recv-Q Send-Q      Local Address           Foreign Address         State
tcp     0          0      0.0.0.0:80              0.0.0.0:*               LISTEN
tcp     0          0      0.0.0.0:21              0.0.0.0:*               LISTEN
tcp     0          0      0.0.0.0:22              0.0.0.0:*               LISTEN
```

Unfortunately, netstat is one of those tools that have different options on different unix distributions, which might require a little manual reading before finding the correct options – *netstat -an* does however appear to be usable many places (linux, bsd, solaris, windows and most likely others too)

$ **netstat -an| grep LISTEN**

In many cases, netstat will be more verbose than it needs to be – this makes a |grep LISTEN useful. In the above example, the interesting line is the first, which tells us that there is a process accepting connections on port 80.

$ **netstat -anp**

On some linux versions of netstat, something like the above will work, adding information about what process is running on which port.

Tools – Other

http://httpd.apache.org/dev/debugging.html

Lists a few additional tools for debugging Apache, but they are beyond the scope of this talk.

1 <!DOCTY should give it away – this is in fact an attempt to connect towww.apache.org on port 80, where there is no HTTPS.

http://www.research.att.com/~hpk/wsp/

The Web Scraping Proxy is a little tool written in perl, which can act as a proxy between a regular browser and the webserver. WSP will output headers and such, which can be used to learn about the exact communication between the server and the client.

Logs - Always check the error log

```
ErrorLog - (logs/error_log)
          LogLevel - (debug, info, notice, warn, error...)
```

The error log should almost always be the first place you look when troubleshooting Apache. While the webserver will present the client with a short error message, there's likely to be additional information in the error log – general security recommendations state that giving too much information away in error situations might help an attacker in gaining knowledge that can be used in an attack. When debugging, you should turn up the LogLevel to debug to get additional information.

```
RewriteLog
          RewriteLogLevel (0 – 9)
```

See the mod_rewrite talk for additional details.

```
ScriptLog
          ScriptLogBuffer (1024)
          ScriptLogLength
```

ScriptLog is not a widely known log, but can be used to get additional information when debugging scripts (usually the ErrorLog will be enough).

```
--with-suexec-log= - (logs/suexec_log)
          AP_LOG_EXEC (suexec -V)
```

The suexec log is the place to look in case of scripting errors when suexec is enabled.

```
SSLLog - 1.3 mod_ssl only
          SSLLogLevel - 1.3 mod_ssl only
```

SSLLog is a legacy from mod_ssl in apache 1.3 – the information that used to go there now goes into the main error log.

Troubleshooting

I've trird to group errors to make it easier to know where to start looking.

1. Connection errors

Remember that browsers lie, but here's a few things to try when the browser claims it can't conn

1. is the server running at all

    ps

    netstat -an

    telnet 127.0.0.1 80

ps – is apache running at all.

netstat – is it listening on the correct port/ip?

telnet – first try locally to determine wether there is a reply there, and then later remotely. If the connection works locally, but not remotely, then look for firewalls and similar problems.

2. did the request reach the server

    tail -f logs/access

    tcpdump

Check the access log to see wether the request makes it to apache? Or use tcpdump to look at the traffic.

3. protocol errors (SSL)

    openssl s_client

Use *openssl s_client* as explained in the tools section.

2. Startup errors

Always look in the error log

strace/truss/trace

The error log will often contain information about why Apache didn't start – if it doesn't, then it is time to look at the debugging hints at http://httpd.apache.org/dev/debugging.html

If you find the error message "Socket Already in use", then something else is using the port you're trying to use – first verify the Listen directive in httpd.conf and then check with ps and netstat to see wether something is already listening. Another tool that can help determine which process that is blocking the use of your network is lsof[2]

Common causes are: ServerName not being set correctly, trying to create logfiles in directories that doesn't exist.

3. 404s

404 not found

error log

access log

The error log will usually tell you what file was requested, and what filename Apache translated the request to – if it doesn't, then make sure you have LogLevel set to debug. If those get you nothing, check the access log for a Redirect and see below.

Common causes

(Script)Alias - (also mismatched /)

See: http://httpd.apache.org/docs/misc/FAQ-E.html#set-servername

Redirects

telnet

ServerName

Use telnet to verify that you're not being redirected to the wrong location.

Relative / Absolute paths

RewriteRules

RewriteLog / RewriteLogLevel 9

Set up a RewriteLog – see "WE11, "Apache mod_rewrite, the Swiss

2 See ftp://vic.cc.purdue.edu/pub/tools/unix/lsof/

Army Knife of URL manipulation" for details.

4. 403s

In case of a *403 Forbidden* always check the error log. Not all information go into the error page.

If you get "client denied by server configuration" in the log, then look through httpd.conf for *deny from.*

If you see "user dfsfdd not found", then check the check password file.

If you see "does not meet 'require'ments for user/valid-user" then check the Require statement – it should be one of valid-user / group / user.

6. Scripting errors

When you see a "*500 Internal server error"*, first check the error log – if it mentions "Premature end of script headers", then execute the script locally "./script" – does it output the correct headers? If you find nothing there, and have suexec enabled, then look in the suexec log.

If you get no output or unparsed output, then make sure that ScriptAlias is set and that you have Options +ExecCGI for the dir in question.

7. htaccess errors

For *"500 Internal server error"*, check the error log as usual – the types of errors that would result in a syntax error if they had been in httpd.conf will give a 500 in htaccess. Common examples are "Not allowed in this context" which means a directive was used that can't be used in an htaccess file – see the documentation for directives and look for the Context line. Another common mistake is not having allowed the type of directive in your AllowOverride – see the Override: line in the documentation.

If your .htaccess appears to have "No effect" – check your AllowOverride

settings – a simple test would be to use the following command:

```
echo "garbage" >> .htaccess
```

If that doesn't give an error on the next request to that location, then you have AllowOverride none in effect.

Final words

I hope this talk has given you some ideas about how to get started on troubleshooting Apache configurations.

This presenation has ended.
Click anywhere to return.

_

